



Projet DATAImmo

Tiphanie RONEL – Data Analyst

1

Contexte

2

Structuration de la base de données

3

Analyse des biens immobiliers

Objectif général : Analyser et prévoir le prix de vente des biens immobiliers

Pourquoi

- Meilleur accompagnement des clients en agence régionale
- Se démarquer de la concurrence

Comment

- Modèle d'analyse des prix avec l'historique des transactions sur X années

Étape actuelle

- Base de données implémentée
- Test analyses sur un semestre d'une année

Table en entrée	Source	Description	Périmètre	Données sensibles?
1. Données communes	INSEE	Population	France et Outre-Mer	Non
2. Référentiel géographique	data.gouv	Historique des communes	France et Outre-Mer	Non
3. Valeurs foncières	explore.data.gouv	Prix de vente des biens, Infos sur les biens	• 1^{er} semestre 2020 • France et Outre-Mer sauf Alsace, Moselle, Mayotte	Nom de l'acquéreur + adresse du bien ; /\ consentement explicite ? + durée conservation des données

Tables nettoyées – Excel, Power Query

Table source	Table en sortie	1NF (1 ^{ère} forme normale)	2NF	3NF	Conformité RGPD
Référentiel géographique	1. Commune	- Clé primaire « comcode » - Division nom et n° département - Modification des types - Vérification doublons	✓ (pas de clé composite → pas de dépendance partielle)	Séparation communes et régions pour éviter redondance des informations.	Minimalisation des données
Référentiel géographique	2. Région	- Clé primaire : id_region - Modification des types - Vérification doublons			
Données communes	3. Population	- Clé primaire: concaténation - Modification des types - Vérification doublons		✓	
Valeurs foncières	4. Bien	- Clé primaire : id_bien - Clé étrangère : concaténation - Modification des types - Vérification doublons		Séparation Bien et Vente pour éviter dépendance transitive des informations sur les biens.	- Minimalisation des données - Anonymisation des données
	5. Vente	- Clé primaire : id_vente - Clé étrangère : id_bien - Modification des types - Vérification doublons			

Puis export au format CSV (Comma Separated Values)

TABLE	CODE	SIGNIFICATION	TYPE	LONGUEUR	NATURE	REGLE DE GESTION	REGLE DE CALCUL
bien	id_bien	Code artificiel identifiant le bien	INTEGER	-	INDEX AJOUTÉ	NON-NULL	code dep & code com
	dep_com	concaténation code département et code commune	VARCHAR	10	COMPOSÉE	NON-NULL	
	type_local	Appartement ou maison	VARCHAR	11	ELEMENTAIRE	NON-NULL	
	surf_reel_bati	surface du bien en m2	INTEGER	-	ELEMENTAIRE	NON-NULL	
	nb_piece	nombre de pièces principales du bien	INTEGER	-	ELEMENTAIRE	NON-NULL	
vente	id_vente	Code artificiel identifiant la transaction	INTEGER	-	INDEX AJOUTÉ	NON-NULL	date sans heure
	date_vente	date de la vente	DATE	-	MODIFIE	JJ/MM/AAAA ; NON-NULL	
	valeur_fonciere_euro	Valeur foncière du bien	FLOAT	-	ELEMENTAIRE	NON NULL	
	id_bien	Code artificiel identifiant le bien	INTEGER	-	INDEX AJOUTÉ	NON-NULL	
commune	dep_com	concaténation code département et code commune. com_code renommé	VARCHAR	10	ELEMENTAIRE	NON-NULL	-
	com_nom	nom de la commune (avec accents et symboles)	VARCHAR	50	ELEMENTAIRE	NON-NULL	-
	dep_nom	nom du département	VARCHAR	50	ELEMENTAIRE	NON-NULL	-
	dep_code	code de département	VARCHAR	5	ELEMENTAIRE	NON-NULL	-
	id_region	Code artificiel identifiant la région	INTEGER	-	ELEMENTAIRE	NON-NULL	-
region	id_region	Code artificiel identifiant la région	INTEGER	-	ELEMENTAIRE	NON-NULL	-
	reg_nom	nom actuel de la région	VARCHAR	50	ELEMENTAIRE	NON-NULL	-
population	dep_com	concaténation code département et code commune. com_code renommé	VARCHAR	10	ELEMENTAIRE	NON-NULL	-
	pop_mun	population municipale : personnes habitant dans la commune (PMUN)	INTEGER	-	ELEMENTAIRE	NON-NULL	-
	pop_cap	population comptée à part : personnes recensées ailleurs mais qui ont conse	INTEGER	-	ELEMENTAIRE	NON-NULL	-
	pop_tot	population totale (PTOT)	INTEGER	-	ELEMENTAIRE	NON-NULL	PMUN + PCAP

Légende :

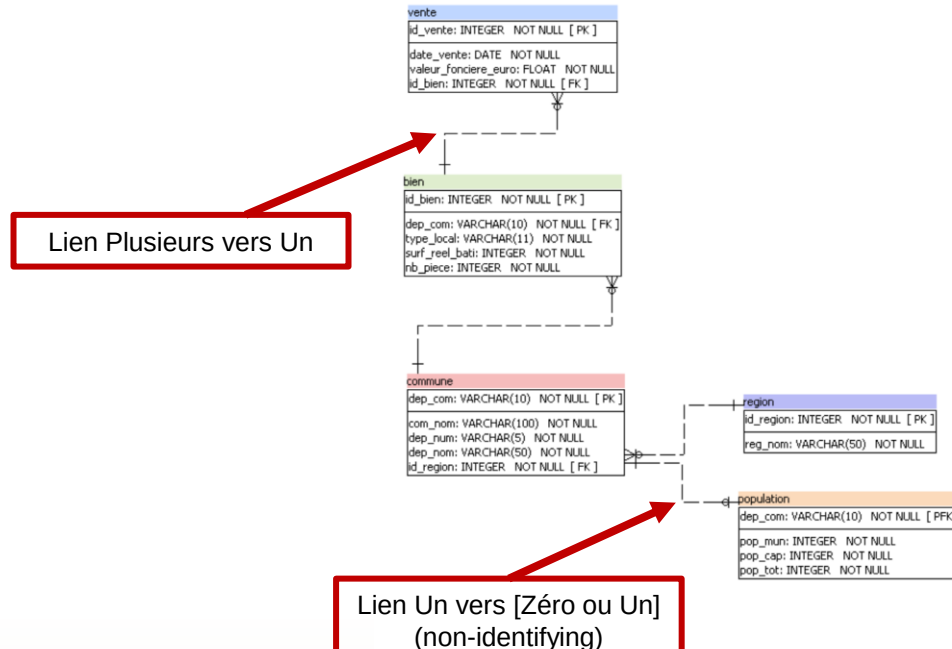
clé primaire

clé étrangère

Requête SQL générée pour créer les tables vides

1

Schématisation des 5 tables



```
CREATE TABLE ps_immo.region (  
    id_region INTEGER NOT NULL,  
    reg_nom VARCHAR(50) NOT NULL,  
    CONSTRAINT id_region PRIMARY KEY (id_region)  
);  
  
CREATE TABLE ps_immo.commune (  
    dep_com VARCHAR(10) NOT NULL,  
    com_nom VARCHAR(100) NOT NULL,  
    dep_num VARCHAR(5) NOT NULL,  
    dep_nom VARCHAR(50) NOT NULL,  
    id_region INTEGER NOT NULL,  
    CONSTRAINT commune_pk PRIMARY KEY (dep_com)  
);  
  
CREATE TABLE ps_immo.population (  
    dep_com VARCHAR(10) NOT NULL,  
    pop_mun INTEGER NOT NULL,  
    pop_cap INTEGER NOT NULL,  
    pop_tot INTEGER NOT NULL,  
    CONSTRAINT population_pk PRIMARY KEY (dep_com)  
);  
  
CREATE TABLE ps_immo.bien (  
    id_bien INTEGER NOT NULL,  
    dep_com VARCHAR(10) NOT NULL,  
    type_local VARCHAR(11) NOT NULL,  
    surf_reel_bati INTEGER NOT NULL,  
    nb_piece INTEGER NOT NULL,  
    CONSTRAINT bien_pk PRIMARY KEY (id_bien)  
);  
  
CREATE TABLE ps_immo.vente (  
    id_vente INTEGER NOT NULL,  
    date_vente DATE NOT NULL,  
    valeur_foncieres_euro REAL NOT NULL,  
    id_bien INTEGER NOT NULL,  
    CONSTRAINT vente_pk PRIMARY KEY (id_vente)  
);  
  
ALTER TABLE ps_immo.commune ADD CONSTRAINT region_commune_fk  
FOREIGN KEY (id_region)  
REFERENCES ps_immo.region (id_region)  
ON DELETE NO ACTION  
ON UPDATE NO ACTION  
NOT DEFERRABLE;  
  
ALTER TABLE ps_immo.bien ADD CONSTRAINT ref_geo_bien_fk  
FOREIGN KEY (dep_com)  
REFERENCES ps_immo.commune (dep_com)  
ON DELETE NO ACTION  
ON UPDATE NO ACTION  
NOT DEFERRABLE;  
  
ALTER TABLE ps_immo.population ADD CONSTRAINT commune_population_fk  
FOREIGN KEY (dep_com)  
REFERENCES ps_immo.commune (dep_com)  
ON DELETE NO ACTION  
ON UPDATE NO ACTION  
NOT DEFERRABLE;  
  
ALTER TABLE ps_immo.vente ADD CONSTRAINT bien_vente_fk  
FOREIGN KEY (id_bien)  
REFERENCES ps_immo.bien (id_bien)  
ON DELETE NO ACTION  
ON UPDATE NO ACTION  
NOT DEFERRABLE;
```

Mise en place de la base de données et import des données - PG Admin, PostgreSQL

1 Création

▼ p5_immo

- > Aggregates
- > Collations
- > Domains
- > FTS Configurations
- > FTS Dictionaries
- > Aa FTS Parsers
- > FTS Templates
- > Foreign Tables
- > Functions
- > Materialized Views
- > Operators
- > Procedures
- > 1.3 Sequences
- ▼ Tables (5)
 - > bien
 - > commune
 - > population
 - > region
 - > vente
- > Trigger Functions
- > Types
- > Views

2 Vérification

Bien

	id_bien [PK] integer	dep_com character varying (10)	type_local character varying (11)	surf_reel_bati integer	nb_piece integer
1	1	01103	Appartement	48	3
2	2	06004	Appartement	40	1
3	3	06088	Appartement	82	3
4	4	06123	Appartement	27	1
5	5	13005	Appartement	47	2

34 169
lignes

Commune

	dep_com [PK] character varying (10)	com_nom character varying (100)	dep_num character varying (5)	dep_nom character varying (50)	id_region integer
1	01001	L'Abergement-Clémenciat	01	Ain	1
2	01002	L'Abergement-de-Varey	01	Ain	1
3	01003	Amareins	01	Ain	1
4	01004	Ambérieu-en-Bugey	01	Ain	1
5	01005	Ambérieu-en-Dombes	01	Ain	1

38 916
lignes

Population

	dep_com [PK] character varying (10)	pop_mun integer	pop_cas integer	pop_tot integer
1	01001	779	19	798
2	01002	256	1	257
3	01004	14134	380	14514
4	01005	1751	25	1776
5	01006	112	6	118

34 991
lignes

Région

	id_region [PK] integer	reg_nom character varying (50)
1	1	Auvergne-Rhône-Alpes
2	2	Hauts-de-France
3	3	Provence-Alpes-Côte d'Azur
4	4	Grand Est
5	5	Occitanie

19
lignes

Vente

	id_vente [PK] integer	date_vente date	valeur_fonciere_euro real	id_bien integer
1	1	2020-01-02	165000	1
2	2	2020-01-02	355680	2
3	3	2020-01-02	229500	3
4	4	2020-01-02	125000	4
5	5	2020-01-02	90000	5

34 169
lignes



Analyse des biens immobiliers

1. Nombre total d'appartements vendus au 1er semestre 2020



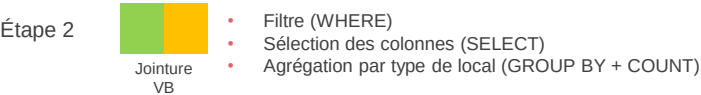
31 378 appartements ont été vendus au 1^{er} semestre 2020

Requête SQL & Résultat brut

```
SELECT
  b.type_local,
  COUNT(b.type_local) as nb_appartement_vendu
FROM p5_immo.vente as v
JOIN p5_immo.bien as b
ON v.id_bien=b.id_bien
WHERE
  b.type_local='Appartement'
  AND v.date_vente >= '01/01/2020'
  AND v.date_vente <= '30/06/2020'
GROUP BY b.type_local;
```

	type_local character varying (11)	nb_appartement_vendu bigint
1	Appartement	31378

Explications de la requête



2. Nombre d'appartements vendus au 1er semestre 2020, par région

TOP 3 des régions avec le plus de ventes



1. Île-de-France

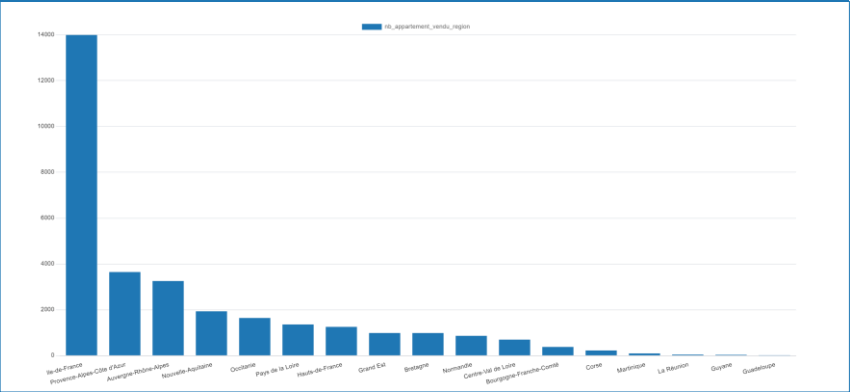
2. Provence-Alpes-Côte d'Azur

3. Auvergne-Rhône-Alpes
- 13 995

3 649

3 253

Classement des régions par nombre de ventes



Résultat brut

	reg_nom character varying (50)	nb_appartement_vendu_region bigint
1	Île-de-France	13995
2	Provence-Alpes-Côte d'Azur	3649
3	Auvergne-Rhône-Alpes	3253
4	Nouvelle-Aquitaine	1932
5	Occitanie	1640
6	Pays de la Loire	1357
7	Hauts-de-France	1254
8	Grand Est	984
9	Bretagne	983
10	Normandie	862
11	Centre-Val de Loire	696
12	Bourgogne-Franche-Comté	376
13	Corse	223
14	Martinique	94
15	La Réunion	44
16	Guyane	34
17	Guadeloupe	2

Requête SQL

```
SELECT
  r.reg_nom,
  COUNT(r.reg_nom) AS nb_appartement_vendu_region
FROM p5_immo.vente AS v
JOIN p5_immo.bien AS b ON v.id_bien=b.id_bien
JOIN p5_immo.commune AS c ON c.dep_com = b.dep_com
JOIN p5_immo.region AS r on r.id_region = c.id_region
WHERE
  b.type_local='Appartement'
  AND v.date_vente >= '01/01/2020'
  AND v.date_vente <= '30/06/2020'
GROUP BY r.reg_nom
ORDER BY nb_appartement_vendu_region DESC;
```

Explications de la requête

Étape 1



Étape 2



- Filtre (WHERE)
- Sélection des colonnes (SELECT)
- Agrégation par région (GROUP BY + COUNT)
- Tri décroissant (ORDER BY...DESC)

3. Proportion des ventes d'appartements en fonction du nombre de pièces

TOP 3 des types d'appartements
les plus vendus



1. Les 2-pièces

2. Les 3-pièces

3. Les 1-pièces

31,2 %

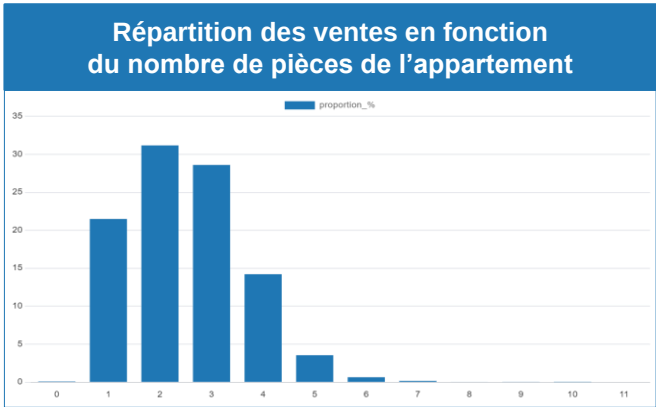
28,6 %

21,5%

9 783 ventes

8 966 ventes

6 739 ventes



Résultat brut

	type_local character varying (11)	nb_piece integer	nb_ventes bigint	proportion numeric
1	Appartement	0	30	0.10
2	Appartement	1	6739	21.48
3	Appartement	2	9783	31.18
4	Appartement	3	8966	28.57
5	Appartement	4	4460	14.21
6	Appartement	5	1114	3.55
7	Appartement	6	204	0.65
8	Appartement	7	54	0.17
9	Appartement	8	17	0.05
10	Appartement	9	8	0.03
11	Appartement	10	2	0.01
12	Appartement	11	1	0.00

Requête SQL

```
WITH total_ventes AS (  
  SELECT  
    COUNT(v.id_vente) AS total_ventes,  
    b.type_local  
  FROM p5_immo.vente AS v  
  JOIN p5_immo.bien AS b ON v.id_bien = b.id_bien  
  WHERE b.type_local = 'Appartement'  
  GROUP BY b.type_local  
)  
ventes_appartements AS (  
  SELECT  
    b.type_local,  
    b.nb_piece,  
    COUNT(v.id_vente) AS nb_ventes  
  FROM p5_immo.vente AS v  
  JOIN p5_immo.bien AS b ON v.id_bien = b.id_bien  
  WHERE b.type_local = 'Appartement'  
  GROUP BY b.nb_piece, b.type_local  
  ORDER BY b.nb_piece  
)  
  
SELECT  
  va.type_local,  
  va.nb_piece,  
  va.nb_ventes,  
  ROUND(CAST(va.nb_ventes * 100.0 / tv.total_ventes AS NUMERIC), 2) AS proportion  
FROM ventes_appartements AS va, total_ventes AS tv  
ORDER BY va.nb_piece;
```

Explications de la requête

Étape 1

vente

bien

Jointure vente-bien

Total ventes

Nombre ventes appartements

Étape 2

Nombre ventes appartements & Total ventes & Calcul proportion

4. Liste des 10 départements où le prix au mètre carré est le plus élevé

TOP 3 des départements
où le prix au m² est le plus élevé



1. Paris

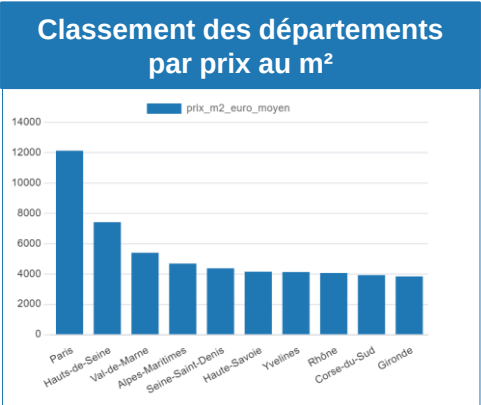
2. Hauts-de-Seine

3. Val-de-Marne

12 K€ / m²

7,4 K€ / m²

5,4 K€ / m²



Résultat brut

	departement character varying (50)	prix_m2_euro_moyen numeric
1	Paris	12121.88
2	Hauts-de-Seine	7415.28
3	Val-de-Marne	5395.98
4	Alpes-Maritimes	4681.76
5	Seine-Saint-Denis	4363.01
6	Haute-Savoie	4149.56
7	Yvelines	4126.19
8	Rhône	4063.83
9	Corse-du-Sud	3905.71
10	Gironde	3838.74

Requête SQL

```
SELECT
  c.dep_nom AS departement,
  ROUND(CAST(AVG(v.valeur_fonciere_euro / b.surf_reel_bati) AS NUMERIC), 2) AS prix_m2_euro_moyen
FROM p5_immo.vente AS v
JOIN p5_immo.bien AS b ON v.id_bien = b.id_bien
JOIN p5_immo.commune AS c ON b.dep_com = c.dep_com
GROUP BY c.dep_nom
ORDER BY prix_m2_euro_moyen DESC
LIMIT 10;
```

Explications de la requête

Étape 1

vente

bien

commune

→

Jointure VBC

Étape 2

Jointure VBC

- Sélection des colonnes (SELECT)
- Agrégation par département (GROUP BY + calcul)
- Tri décroissant (ORDER BY...DESC)
- Top 10 (LIMIT)

5. Prix moyen au mètre carré d'une maison en Île-de-France



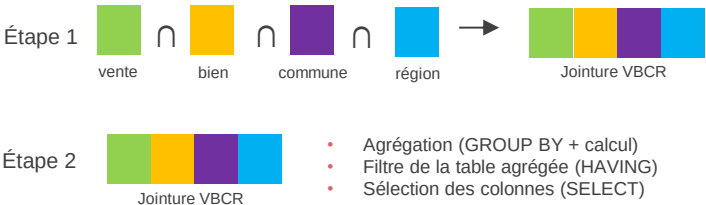
3 998 € : c'est le le prix moyen au m² pour une maison en Île-de-France.

Requête SQL & Résultat brut

```
SELECT
  r.reg_nom AS region,
  b.type_local,
  ROUND(CAST(AVG(v.valeur_fonciere_euro /b.surf_reel_bati) AS NUMERIC), 2) AS prix_m2_euro
FROM p5_immo.vente AS v
JOIN p5_immo.bien AS b ON v.id_bien = b.id_bien
JOIN p5_immo.commune AS c ON b.dep_com = c.dep_com
JOIN p5_immo.region AS r ON r.id_region = c.id_region
GROUP BY r.reg_nom, b.type_local
HAVING
  r.reg_nom = 'Ile-de-France'
  AND b.type_local = 'Maison';
```

	region character varying (50)	type_local character varying (11)	prix_m2_euro numeric
1	Ile-de-France	Maison	3997.71

Explications de la requête



6. Liste des 10 appartements les plus chers avec la région et le nombre de mètres carrés

Région la plus chère pour les appartements



Île-de-France

TOP 3 des appartements les plus chers

€€€€

1.

9M €

10 m²

2.

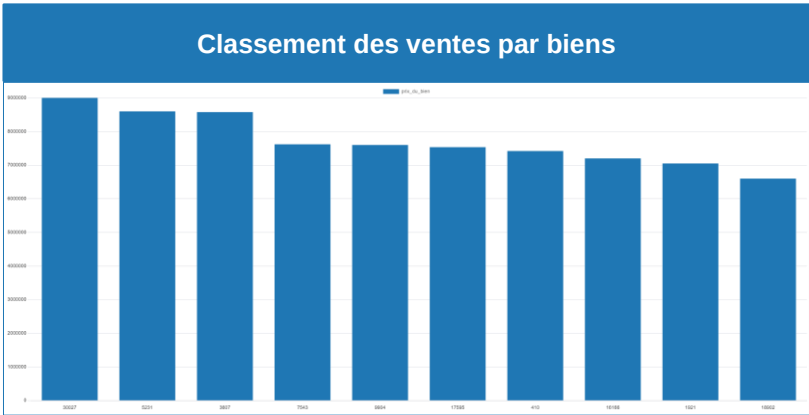
8,6M €

62 m²

3.

8,58M €

289 m²



Résultat brut

	id_bien integer	region character varying (50)	surface_m2 integer	prix_du_bien real
1	30603	Ile-de-France	10	9e+06
2	5261	Ile-de-France	62	8.6e+06
3	3625	Ile-de-France	289	8.577713e+06
4	7602	Ile-de-France	42	7.62e+06
5	9988	Ile-de-France	200	7.6e+06
6	17823	Ile-de-France	143	7.535e+06
7	410	Ile-de-France	357	7.42e+06
8	16357	Ile-de-France	241	7.2e+06
9	1924	Ile-de-France	310	7.05e+06
10	19161	Ile-de-France	76	6.6e+06

Requête SQL

```
SELECT
  b.id_bien,
  r.reg_nom AS region,
  b.surf_reel_bati AS surface_m2,
  v.valeur_fonciere_euro AS prix_du_bien
FROM p5_immo.vente AS v
JOIN p5_immo.bien AS b ON v.id_bien = b.id_bien
JOIN p5_immo.commune AS c ON b.dep_com = c.dep_com
JOIN p5_immo.region AS r ON r.id_region = c.id_region
WHERE
  b.type_local = 'Appartement'
ORDER BY v.valeur_fonciere_euro DESC
LIMIT 10;
```

Explications de la requête

Étape 1

vente

bien

commune

région

→

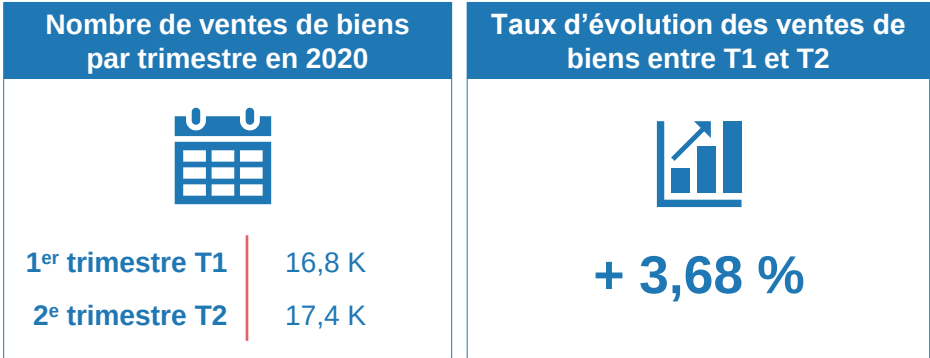
Jointure VBCR

Étape 2

Jointure VBCR

- Filtre (WHERE)
- Sélection des colonnes (SELECT)
- Tri décroissant (ORDER BY...DESC)
- Top 10 (LIMIT)

7. Taux d'évolution du nombre de ventes entre le premier et le second trimestre de 2020



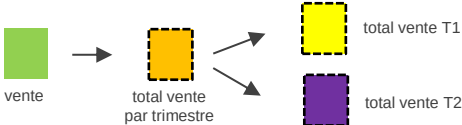
Requête SQL & Résultat brut

```
WITH ventes_par_trimestre AS (  
  SELECT  
    EXTRACT(QUARTER FROM v.date_vente) AS trimestre,  
    EXTRACT(YEAR FROM v.date_vente) AS annee,  
    COUNT(v.date_vente) AS nombre_ventes  
  FROM p5_immo.vente AS v  
  WHERE EXTRACT(YEAR FROM v.date_vente) = 2020  
  GROUP BY trimestre, annee  
)  
ventes_t1 AS (SELECT nombre_ventes AS ventes_t1 FROM ventes_par_trimestre WHERE trimestre = 1),  
ventes_t2 AS (SELECT nombre_ventes AS ventes_t2 FROM ventes_par_trimestre WHERE trimestre = 2)  
  
SELECT  
  ventes_t1,  
  ventes_t2,  
  ROUND(((ventes_t2 - ventes_t1) * 100.0 / ventes_t1), 2) AS "taux_evolution_%"  
FROM ventes_t1 AS t1, ventes_t2 AS t2
```

	ventes_t1 bigint	ventes_t2 bigint	taux_evolution_% numeric	
1	16776	17393	3.68	

Explications de la requête

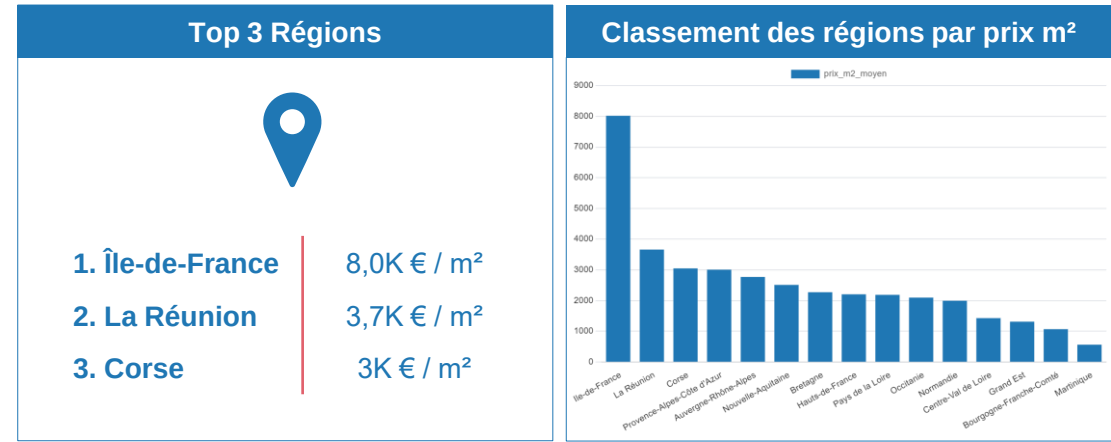
Étape 1



Étape 2



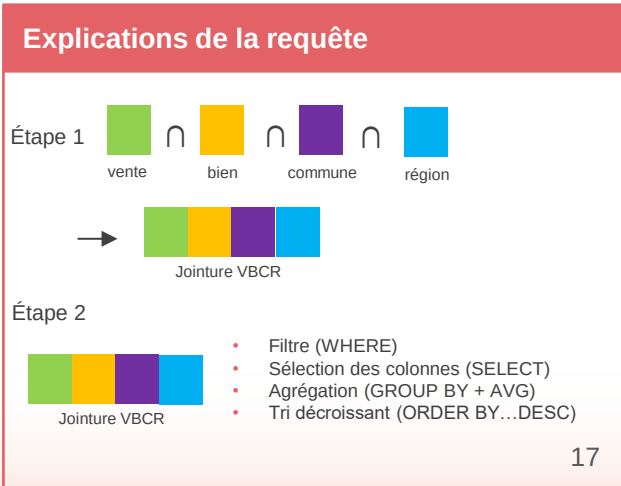
8. Classement des régions en fonction du prix au mètre carré des appartements de plus de 4 pièces



Résultat brut			
region	type_local	prix_m2_moyen	
character varying (50)	character varying (11)	numeric	
1	Île-de-France	Appartement	8003.39
2	La Réunion	Appartement	3659.83
3	Corse	Appartement	3046.47
4	Provence-Alpes-Côte d'Azur	Appartement	3005.24
5	Auvergne-Rhône-Alpes	Appartement	2768.87
6	Nouvelle-Aquitaine	Appartement	2510.18
7	Bretagne	Appartement	2271.86
8	Hauts-de-France	Appartement	2203.61
9	Pays de la Loire	Appartement	2186.72
10	Occitanie	Appartement	2096.42
11	Normandie	Appartement	1994.25
12	Centre-Val de Loire	Appartement	1428.51
13	Grand Est	Appartement	1313.26
14	Bourgogne-Franche-Comté	Appartement	1068.93
15	Martinique	Appartement	564.22

Requête SQL

```
SELECT
  r.reg_nom AS region,
  b.type_local,
  ROUND(CAST(AVG(v.valeur_fonciere_euro / b.surf_reel_bati) AS NUMERIC), 2) AS prix_m2_moyen
FROM p5_immo.vente AS v
JOIN p5_immo.bien AS b ON v.id_bien = b.id_bien
JOIN p5_immo.commune AS c ON b.dep_com = c.dep_com
JOIN p5_immo.region AS r ON r.id_region = c.id_region
WHERE
  b.nb_piece > 4
  AND b.type_local='Appartement'
GROUP BY r.reg_nom, type_local
ORDER BY prix_m2_moyen DESC;
```



9. Liste des communes ayant eu au moins 50 ventes au 1er trimestre

Top 3 communes avec au moins 50 ventes au T1 2020



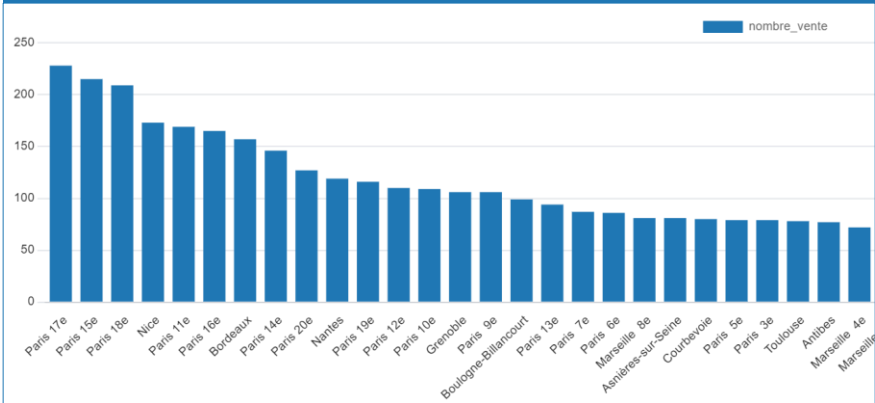
1. Paris 17e

228 ventes
2. Paris 15e

215 ventes
3. Paris 18e

209 ventes

Classement des communes par nombre de ventes, T1 2020



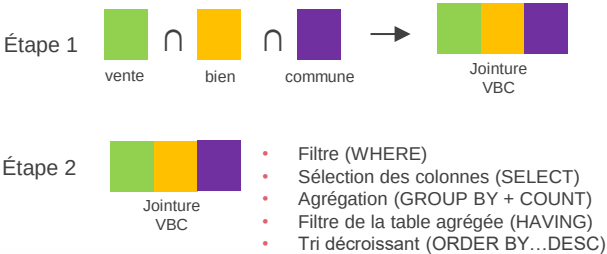
Résultat brut (extrait)

	nom_commune character varying (100)	nombre_vente bigint
1	Paris 17e	228
2	Paris 15e	215
3	Paris 18e	209
4	Nice	173
5	Paris 11e	169
6	Paris 16e	165
7	Bordeaux	157
8	Paris 14e	146
9	Paris 20e	127
10	Nantes	119
11	Paris 19e	116

Requête SQL

```
SELECT
  c.com_nom AS nom_commune,
  COUNT(c.com_nom) AS nombre_vente
FROM p5_immo.vente AS v
JOIN p5_immo.bien AS b
ON v.id_bien = b.id_bien
JOIN p5_immo.commune AS c
ON b.dep_com = c.dep_com
WHERE
  v.date_vente >= '01/01/2020'
  AND v.date_vente <= '31/03/2020'
GROUP BY c.com_nom
HAVING COUNT(c.com_nom) >= 50
ORDER BY nombre_vente DESC;
```

Explications de la requête



10. Appartement 2 pièces vs 3 pièces : Différence en pourcentage du prix au mètre carré

Comparaison prix au m²		Différence en %
2-pièces	4,9K € / m²	 -13 %
3-pièces	4,3K € / m²	

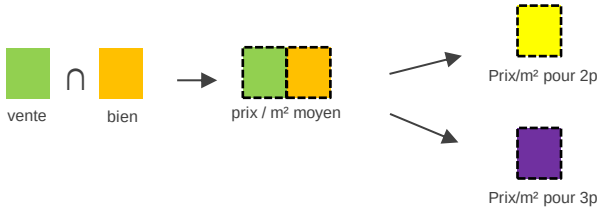
Requête SQL & Résultat brut

```
WITH prix_moyen_au_m2 AS (  
  SELECT  
    b.nb_piece,  
    ROUND(CAST(AVG(v.valeur_fonciere_euro / b.surf_reel_bati) AS NUMERIC),2) AS prix_m2_moyen  
  FROM p5_immo.vente AS v  
  JOIN p5_immo.bien AS b ON v.id_bien = b.id_bien  
  WHERE  
    b.type_local = 'Appartement'  
    AND b.nb_piece IN (2, 3)  
  GROUP BY b.nb_piece  
)  
prix_2p_au_m2 AS(SELECT prix_m2_moyen AS prix_2p FROM prix_moyen_au_m2 WHERE nb_piece = 2),  
prix_3p_au_m2 AS(SELECT prix_m2_moyen AS prix_3p FROM prix_moyen_au_m2 WHERE nb_piece = 3)  
  
SELECT  
  prix_2p,  
  prix_3p,  
  ROUND(CAST((prix_3p - prix_2p) * 100.0 / prix_2p AS NUMERIC), 2) AS diff_pourcentage  
FROM prix_2p_au_m2, prix_3p_au_m2;
```

	prix_2p numeric 🔒	prix_3p numeric 🔒	diff_pourcentage numeric 🔒
1	4927.80	4285.37	-13.04

Explications de la requête

Étape 1



vente bien prix / m² moyen Prix/m² pour 2p Prix/m² pour 3p

Étape 2



Prix/m² 2p & Prix/m² 3p & Calcul différence

11. Moyennes de valeurs foncières pour le top 3 des communes des départements 06, 13, 33, 59 et 69

- 06 - Alpes-Maritimes
- 13 - Bouches-du-Rhône
- 33 - Gironde
- 59 - Nord
- 69 - Rhône

Top 3 communes avec les moyennes les plus hautes



1. Saint-Jean-Cap-Ferret (06)

2. Èze (06)

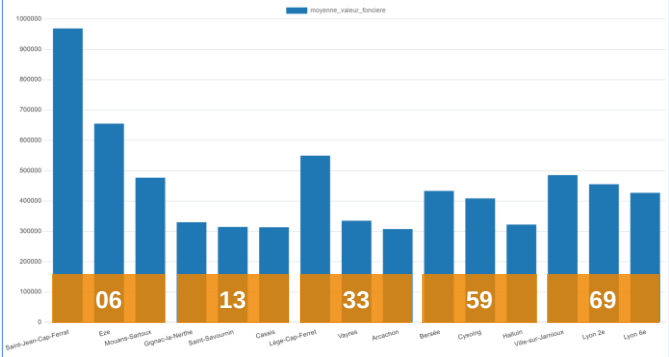
3. Lège-Cap-Ferret (33)

968,8 K€

655 K€

549,5 K€

Moyenne des valeurs foncières par communes



Commune	Dpt	Moyenne valeur (K€)
Saint-Jean-Cap-Ferret	06	968,8
Èze	06	655
Mouans-Sartoux	06	476,9
Gignac-la-Nerthe	13	330,0
Saint-Savournin	13	314,2
Cassis	13	313,4
Lège-Cap-Ferret	33	549,5
Vayres	33	335,0
Arcachon	33	307,4
Bersée	59	433,2
Cysoing	59	408,5
Halluin	59	322,2
Ville-sur-Jarniou	69	485,3
Lyon 2e	69	455,2
Lyon 6e	69	426,9

Résultat brut

	dep_num character varying (5)	com_nom character varying (100)	moyenne_valeur_fonciere numeric	rang bigint
1	06	Saint-Jean-Cap-Ferret	968750.00	1
2	06	Èze	655000.00	2
3	06	Mouans-Sartoux	476898.09	3
4	13	Gignac-la-Nerthe	330000.00	1
5	13	Saint-Savournin	314425.00	2
6	13	Cassis	313416.88	3
7	33	Lège-Cap-Ferret	549500.64	1
8	33	Vayres	335000.00	2
9	33	Arcachon	307435.93	3
10	59	Bersée	433202.00	1
11	59	Cysoing	408550.00	2
12	59	Halluin	322250.00	3
13	69	Ville-sur-Jarniou	485300.00	1
14	69	Lyon 2e	455217.27	2
15	69	Lyon 6e	426968.25	3

Requête SQL

```
WITH moyennes_communes AS (  
  SELECT  
    c.dep_num,  
    c.com_nom,  
    ROUND(CAST(AVG(v.valeur_fonciere_euro) AS NUMERIC), 2) AS moyenne_valeur_fonciere  
  FROM p5_immo.vente AS v  
  JOIN p5_immo.bien AS b ON v.id_bien = b.id_bien  
  JOIN p5_immo.commune AS c ON b.dep_com = c.dep_com  
  WHERE c.dep_num IN ('06', '13', '33', '59', '69')  
  GROUP BY c.dep_num, c.com_nom  
),  
ranked_communes AS (  
  SELECT  
    dep_num,  
    com_nom,  
    moyenne_valeur_fonciere,  
    RANK() OVER (PARTITION BY dep_num ORDER BY moyenne_valeur_fonciere DESC) AS rang  
  FROM moyennes_communes  
)  
SELECT  
  dep_num,  
  com_nom,  
  moyenne_valeur_fonciere,  
  rang  
FROM ranked_communes  
WHERE rang <= 3;
```

Explications de la requête

Étape 1

vente

bien

commune

→

Moyennes valeurs foncières dpt

Ranked communes

Étape 2

Ranked communes

→

Ranked communes filtré

- Filtre sur le rang (WHERE)
- Sélection des colonnes (SELECT)

20

12. Top 20 des communes d'au moins 10 000 habitants avec le plus de ventes pour 1000 habitants

Top 3 communes



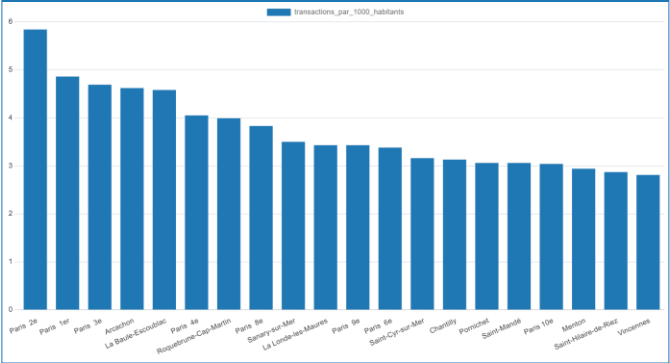
1. Paris 2^e

5,8 ventes pour 1000 habitants
2. Paris 1^{er}

4,9 ventes pour 1000 habitants
3. Paris 3^e

4,7 ventes pour 1000 habitants

Classement des communes par nombre de ventes



Résultat brut (top 20)

	id_region character varying (10)	com_nom character varying (100)	nb_transactions bigint	pop_tot integer	transactions_par_1000_habitants numeric
1	75102	Paris 2e	127	21735	5.84
2	75101	Paris 1er	79	16055	4.92
3	75103	Paris 3e	161	34306	4.69
4	33009	Arcachon	55	11898	4.62
5	44055	La Baule-Escoubiac	77	16797	4.59
6	75104	Paris 4e	120	29390	4.08
7	06104	Roquebrune-Cap-Martin	52	13041	3.99
8	75108	Paris 8e	139	36250	3.83
9	83123	Sanary-sur-Mer	60	17160	3.50
10	83071	La Londe-les-Maures	37	10776	3.43
11	75109	Paris 9e	208	60563	3.43
12	75106	Paris 6e	139	41171	3.38
13	83112	Saint-Cyr-sur-Mer	38	11725	3.24
14	60141	Chantilly	35	11178	3.13
15	44132	Pornichet	35	11440	3.06
16	94067	Saint-Mandé	69	22576	3.06
17	75110	Paris 10e	264	86863	3.04
18	06083	Menton	91	30981	2.94
19	85226	Saint-Hilaire-de-Riez	33	11501	2.87
20	94080	Vincennes	141	50230	2.81

Requête SQL

```
SELECT
  c.dep_com,
  c.com_nom,
  COUNT(v.id_vente) AS nb_transactions,
  p.pop_tot,
  ROUND(CAST((COUNT(v.id_vente) * 1000.0) / p.pop_tot AS NUMERIC), 2) AS transactions_par_1000_habitants
FROM p5_immo.vente AS v
JOIN p5_immo.bien AS b ON v.id_bien = b.id_bien
JOIN p5_immo.commune AS c ON b.dep_com = c.dep_com
JOIN p5_immo.population AS p ON c.dep_com = p.dep_com
WHERE p.pop_tot >= 10000
GROUP BY c.dep_com, c.com_nom, p.pop_tot
ORDER BY transactions_par_1000_habitants DESC
LIMIT 20;
```

Explications de la requête

Étape 1



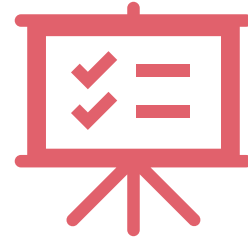
Étape 2



- Filtre (WHERE)
- Sélection des colonnes (SELECT)
- Agrégation (GROUP BY + COUNT & ROUND)
- Tri décroissant (ORDER BY...DESC)
- Top 20 (LIMIT)

Conclusion

- Mise en place de la BDD
- Requêtes SQL
- Analyses





Merci.

Tiphanie RONEl